TITLE: HarborIQ
VERSION: 0.1.0
Source Code Deposit Copy -- Prepared for U.S. Copyright Office
registration under the trade-secret deposit option: first 10 and
last 10 pages (40 lines/page equivalent, per Circular 61 /
37 C.F.R. 202.20(c)(2)(vii)(A)(1)).
Total lines in full combined source listing: 31281
This is a partial deposit by design -- it is NOT the complete program.

==
FIRST 10 PAGES (400 LINES)
==
===== FILE: app/__init__.py =====

===== FILE: app/api/__init__.py =====

===== FILE: app/api/deps.py =====
"""Shared FastAPI dependencies.

Tenant context is derived from a verified Bearer access token — never from a
client-supplied header. `get_current_user` decodes the JWT, re-reads the user
row under RLS, and arms `app.current_company_id` for the rest of the request,
so the existing TenantContext/RLS machinery is driven by authenticated input.

The token carries a `role` claim, but authorization reads the role from the
database row instead. That costs one indexed lookup per request and buys
immediate effect for deactivations and role changes, rather than waiting for
the access token to expire.
"""
from __future__ import annotations

import uuid
from collections.abc import Callable
from dataclasses import dataclass
from datetime import datetime

from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.security import InvalidToken, decode_access_token
from app.core.token_denylist import is_denylisted, is_user_access_epoch_stale
from app.db.models import OPERATIONS_ROLES, USER_MANAGEMENT_ROLES, UserRole
from app.db.session import get_db, get_service_db
from app.db.tenant import set_tenant
from app.services.auth import AuthenticatedUser, load_user

# auto_error=False so a missing header produces our own 401 shape rather than
# FastAPI's 403.
_bearer_scheme = HTTPBearer(auto_error=False, description="Bearer access token")

@dataclass(frozen=True)
class Principal:
    """The authenticated caller plus the session its access token came from."""

    user: AuthenticatedUser
    session_id: uuid.UUID
    token_jti: str
    token_expires_at: datetime
    token_issued_at: datetime

def _unauthenticated(detail: str = "not authenticated") -> HTTPException:
    return HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail=detail,
        headers={"WWW-Authenticate": "Bearer"},
)

```
def get_current_principal(
    credentials: HTTPAuthorizationCredentials | None = Depends(_bearer_scheme),
    db: Session = Depends(get_db),
) -> Principal:
    """Verify the Bearer token, load the user under RLS, and arm the tenant GUC."""
    if credentials is None or not credentials.credentials:
        raise _unauthenticated()

    try:
        claims = decode_access_token(credentials.credentials)
    except InvalidToken as exc:
        raise _unauthenticated(f"invalid access token: {exc}") from exc

    if is_denylisted(claims.jti):
        raise _unauthenticated("access token has been revoked")

    if is_user_access_epoch_stale(claims.user_id, claims.access_epoch):
        raise _unauthenticated("access token has been revoked")

    user = load_user(db, claims.company_id, claims.user_id)
    if user is None or not user.is_active:
        raise _unauthenticated("account is not active")

    set_tenant(db, user.company_id)
    return Principal(
        user=user,
        session_id=claims.session_id,
        token_jti=claims.jti,
        token_expires_at=claims.expires_at,
        token_issued_at=claims.issued_at,
    )


def get_current_user(
    principal: Principal = Depends(get_current_principal),
) -> AuthenticatedUser:
    return principal.user


def get_current_company_id(
    user: AuthenticatedUser = Depends(get_current_user),
) -> uuid.UUID:
    """The acting tenant, derived from the verified token."""
    return user.company_id


def require_roles(*roles: UserRole) -> Callable[..., AuthenticatedUser]:
    """Dependency factory enforcing that the caller holds one of `roles`."""
    allowed = {UserRole(role).value for role in roles}

    def dependency(
        user: AuthenticatedUser = Depends(get_current_user),
    ) -> AuthenticatedUser:
        if user.role not in allowed:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"requires one of these roles: {sorted(allowed)}",
            )
        return user

    return dependency


#: Only owners and admins may provision other users.
require_user_manager = require_roles(*USER_MANAGEMENT_ROLES)

#: Alias for owner/admin-gated billing-admin actions (Stripe Connect
#: onboarding, running the dunning sweep on demand). Same role set as
#: `require_user_manager` -- kept as a separate name because "who may manage
#: teammates" and "who may manage billing/payments configuration" are
#: different concerns that happen to share a role set today; a future phase
#: could split them (e.g. a dedicated billing-admin role) without touching
#: user-management call sites.
require_admin = require_roles(*USER_MANAGEMENT_ROLES)
```

```
#: Front-of-house work — the customer book, the fleet, and dispatching jobs.
#: Technicians are excluded here and are granted narrower access per-endpoint.
require_operations = require_roles(*OPERATIONS_ROLES)

#: Role values that may act on any job in the tenant. Compared against
#: `AuthenticatedUser.role`, which is a plain string.
OPERATIONS_ROLE_VALUES = frozenset(role.value for role in OPERATIONS_ROLES)


def authorize_job_action(
    db: Session, user: AuthenticatedUser, job_id: uuid.UUID
) -> None:
    """Allow the action if the caller runs the shop or owns this work order.

    Owner/admin/office short-circuit without a query. For anyone else the job is
    loaded under RLS first, so a technician probing another tenant's id gets the
    same 404 as for an id that does not exist, rather than a 403 that would
    confirm the job is real.
    """
    if user.role in OPERATIONS_ROLE_VALUES:
        return

    # Imported here: the jobs service pulls in the state machines and schemas,
    # and deps is imported by every route module.
    from app.api.errors import http_errors
    from app.services import jobs as jobs_service

    with http_errors():
        job = jobs_service.get(db, user.company_id, job_id)
    if job.technician_id != user.id:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="only the assigned technician or an office user may act on this job",
        )


__all__ = [
    "OPERATIONS_ROLE_VALUES",
    "Principal",
    "authorize_job_action",
    "get_current_company_id",
    "get_current_principal",
    "get_current_user",
    "get_db",
    "get_service_db",
    "require_admin",
    "require_operations",
    "require_roles",
    "require_user_manager",
]

# ===== FILE: app/api/errors.py =====
"""Translation of service-layer domain errors into HTTP responses.

The services raise domain exceptions and know nothing about HTTP; the routes
own the status codes. Keeping the mapping in one place means every endpoint
answers the same way for the same failure, which matters most for `NotFound`:
under RLS, "belongs to another tenant" is indistinguishable from "does not
exist", and both must be a 404 that leaks nothing.
"""
from __future__ import annotations

from collections.abc import Iterator
from contextlib import contextmanager

from fastapi import HTTPException, status

from app.services.auth import AccountLocked
from app.services.billing import ConnectUnavailable
from app.services.crud import Conflict, NotFound, ValidationFailed
from app.services.invoices import RefundFailed
from app.services.jobs import InvalidTechnician
from app.services.state_machines import IllegalTransition
```

```
#: Checked in order, so a subclass must precede its base.
#: 423 Locked (WebDAV, RFC 4918 §11.3 — repurposed here as the generic "this
#: resource is temporarily locked" status) is what an account-lockout error
#: maps to, distinct from 401 (bad credentials) so a client can tell the two
#: apart without the response body revealing anything about remaining
#: attempts or account existence.
_HTTP_STATUS: tuple[tuple[type[Exception], int], ...] = (
    (NotFound, status.HTTP_404_NOT_FOUND),
    (ValidationFailed, status.HTTP_422_UNPROCESSABLE_CONTENT),
    (InvalidTechnician, status.HTTP_422_UNPROCESSABLE_CONTENT),
    (IllegalTransition, status.HTTP_409_CONFLICT),
    (Conflict, status.HTTP_409_CONFLICT),
    (AccountLocked, status.HTTP_423_LOCKED),
    # Stripe rejected/could not process the refund -- an upstream failure,
    # not a client input error, so 502 rather than 4xx.
    (RefundFailed, status.HTTP_502_BAD_GATEWAY),
    # Same reasoning for Connect onboarding: Stripe unconfigured/unreachable
    # is an upstream failure, not a bad request.
    (ConnectUnavailable, status.HTTP_502_BAD_GATEWAY),
)


@contextmanager
def http_errors() -> Iterator[None]:
    """Re-raise domain exceptions as HTTPExceptions with the agreed status."""
    try:
        yield
    except tuple(exc for exc, _ in _HTTP_STATUS) as exc:
        for kind, code in _HTTP_STATUS:
            if isinstance(exc, kind):
                raise HTTPException(code, str(exc)) from exc
        raise  # unreachable: the except clause is built from the same table

# ===== FILE: app/api/middleware.py =====
"""Cross-cutting ASGI middleware: request-id correlation and Prometheus metrics.

Both middlewares are deliberately dependency-free of the route layer (they
wrap every request at the ASGI level via Starlette's BaseHTTPMiddleware) so
adding a new route never has to remember to opt in.
"""
from __future__ import annotations

import time
import uuid

import structlog
from prometheus_client import Counter, Histogram
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.types import ASGIApp

from app.core.config import settings

REQUEST_ID_HEADER = "X-Request-ID"

log = structlog.get_logger()

# --- Prometheus metrics -----------------------------------------------------
# Labelled by method/path/status. `path` uses the *route template*
# (e.g. "/api/v1/jobs/{id}"), not the raw URL, so per-tenant/per-id traffic
# does not explode the metric's cardinality — see `_route_label` below.
REQUEST_COUNT = Counter(
    "http_requests_total",
    "Total HTTP requests",
    ["method", "path", "status"],
)
REQUEST_LATENCY = Histogram(
    "http_request_duration_seconds",
    "HTTP request latency in seconds",
    ["method", "path", "status"],
)
```

```
def _route_label(request: Request) -> str:
    """Best-effort route template for metric labels.

    Starlette only populates `request.scope["route"]` once routing has
    resolved (i.e. after `call_next` returns), so this must be read after
    the downstream handler runs. Unmatched paths (404s) fall back to the
    raw path — bounded in practice since there is no user-controlled path
    segment that isn't itself templated.
    """
    route = request.scope.get("route")
    if route is not None and getattr(route, "path", None):
        return route.path
    return request.url.path


class RequestIDMiddleware(BaseHTTPMiddleware):
    """Assign (or propagate) a request id and bind it into structlog context.

    - If the inbound request already carries `X-Request-ID` (typically set
      by a reverse proxy / load balancer upstream of this app), that value
      is reused verbatim so a single request keeps one id end to end.
    - Otherwise a fresh UUID4 is generated.
    - The id is bound into `structlog.contextvars` for the lifetime of the
      request, so every log line emitted while handling it — from any
      module, without threading the value through function signatures —
      carries `request_id` automatically.
    - The id is always echoed back as the `X-Request-ID` response header,
      so a caller (or the proxy) can correlate a specific response with the
      server-side logs for it.
    """

    def __init__(self, app: ASGIApp) -> None:
        super().__init__(app)

    async def dispatch(self, request: Request, call_next):
        incoming = request.headers.get(REQUEST_ID_HEADER)
        request_id = incoming if incoming else str(uuid.uuid4())

        structlog.contextvars.clear_contextvars()
        structlog.contextvars.bind_contextvars(request_id=request_id)
        try:
            response = await call_next(request)
        finally:
            structlog.contextvars.clear_contextvars()

        response.headers[REQUEST_ID_HEADER] = request_id
        return response


class SecurityHeadersMiddleware(BaseHTTPMiddleware):
    """Baseline HTTP security response headers (Security Core Prompt v1.0,
    H-3 fix; OWASP ASVS V14 "HTTP Security Headers").

    This is a pure API backend (the SPA is served separately by nginx, which
    gets its own copy of these headers — see `frontend/nginx.conf`), so the
    policy here is deliberately strict and has none of a traditional
    server-rendered site's inline-script/style needs:

    - `Strict-Transport-Security`: only sent once `app_env` is not
      `development`, so a local plain-HTTP dev server is never told by the
      browser to upgrade every future request to HTTPS (which would break
      it). `includeSubDomains` + a 1-year max-age is the standard
      preload-eligible baseline; `preload` itself is intentionally left out
      since submitting to the HSTS preload list is a one-way, cross-team
      decision that shouldn't be made implicitly by this middleware.
    - `Content-Security-Policy`: `default-src 'none'` — this origin never
      serves HTML/JS/images itself (JSON API only), so there is nothing to
      allow-list. `frame-ancestors 'none'` blocks this API from being
      framed anywhere (defense-in-depth alongside `X-Frame-Options`).
    - `X-Content-Type-Options: nosniff` — stops browsers from MIME-sniffing
      JSON responses (or user-uploaded attachment bytes, see M-3) as HTML.
    - `X-Frame-Options: DENY` and `Referrer-Policy: strict-origin-when-
      cross-origin` — standard clickjacking / referrer-leak hardening.
    - `Permissions-Policy` — explicitly denies browser features this API
```

```
      has no reason to grant.
    """

    def __init__(self, app: ASGIApp) -> None:
        super().__init__(app)

    # Swagger UI / ReDoc render actual HTML in the browser and pull their
    # JS/CSS from a CDN, so the `default-src 'none'` API policy below would
    # break them outright. They're documentation surfaces, not the JSON API
    # surface this policy protects, so they get a separate, docs-appropriate
    # CSP instead of being silently broken.
    _DOCS_PATHS = {"/docs", "/redoc", "/openapi.json"}

    async def dispatch(self, request: Request, call_next):
        response = await call_next(request)
        headers = response.headers
        headers["X-Content-Type-Options"] = "nosniff"
        headers["X-Frame-Options"] = "DENY"
        headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
        headers["Permissions-Policy"] = (
            "geolocation=(), camera=(), microphone=(), payment=()"
        )
        if request.url.path in self._DOCS_PATHS:
            headers["Content-Security-Policy"] = (
                "default-src 'self'; img-src 'self' data: fastapi.tiangolo.com; "
                "script-src 'self' cdn.jsdelivr.net 'unsafe-inline'; "
                "style-src 'self' cdn.jsdelivr.net 'unsafe-inline'; "
                "frame-ancestors 'none'"
            )
        else:
            headers["Content-Security-Policy"] = (
                "default-src 'none'; frame-ancestors 'none'"
            )
        if settings.app_env != "development":
            headers["Strict-Transport-Security"] = (
                "max-age=31536000; includeSubDomains"
            )
        return response


class PrometheusMiddleware(BaseHTTPMiddleware):
    """Record request count + latency for every request, labelled by route.

    Deliberately does not special-case `/metrics` itself — scraping the
    scraper endpoint is cheap and consistent, and excluding it would just be

========================================================================
[... 30481 lines omitted per trade-secret deposit rules ...]
========================================================================

========================================================================
LAST 10 PAGES (400 LINES)
========================================================================
}

export interface PortalInviteResponse {
  customer_id: string;
  outbox_event_id: number;
}

// --- Customer <-> staff messaging (Phase 9) ---
// Mirrors app/schemas/messages.py exactly.

export type MessageSenderType = "customer" | "staff";

export interface MessageCreate {
  body: string;
  job_id?: string | null;
}

export interface Message {
  id: string;
  company_id: string;
  customer_id: string;
```

```
  job_id: string | null;
  sender_type: MessageSenderType;
  sender_user_id: string | null;
  body: string;
  // 'portal' or 'sms' (migration 0010, Phase 11) -- how this message arrived.
  channel: "portal" | "sms";
  created_at: string;
  read_at: string | null;
}

export interface StaffMessageCreate extends MessageCreate {
  customer_id: string;
}

export interface InboxMessage extends Message {
  customer_label: string | null;
}

// --- Stripe Connect + dunning (Phase 8) ---
// Mirrors app/schemas/billing.py exactly.

export interface ConnectOnboardingResponse {
  account_id: string;
  onboarding_url: string;
}

export interface ConnectStatusResponse {
  connected: boolean;
  account_id: string | null;
  charges_enabled: boolean;
  details_submitted: boolean;
}

export interface DunningRunResponse {
  reminded_invoice_ids: string[];
  count: number;
}

// --- AR aging report (Phase 8) ---
// Mirrors app/schemas/reports.py exactly.

export interface AgingBuckets {
  current: string;
  days_1_30: string;
  days_31_60: string;
  days_61_90: string;
  days_90_plus: string;
}

export interface CustomerAging {
  customer_id: string | null;
  customer_name: string;
  buckets: AgingBuckets;
  total: string;
  invoice_count: number;
}

export interface ArAgingReport {
  as_of: string;
  customers: CustomerAging[];
  bucket_totals: AgingBuckets;
  grand_total: string;
}

// --- P&L and cash-flow reports (Phase 14) ---
// Mirrors app/schemas/reports.py exactly.

export interface PnlMonth {
  month: string; // "YYYY-MM"
  revenue: string;
  refunds: string;
  net_revenue: string;
  parts_cost: string;
  labor_cost: string;
```

```
  labor_cost_unavailable: boolean;
  unrated_technicians: string[];
  net: string;
}

export interface PnlTotals {
  revenue: string;
  refunds: string;
  net_revenue: string;
  parts_cost: string;
  labor_cost: string;
  labor_cost_unavailable: boolean;
  unrated_technicians: string[];
  net: string;
}

export interface PnlReport {
  start_date: string;
  end_date: string;
  months: PnlMonth[];
  totals: PnlTotals;
}

export interface CashFlowMonth {
  month: string;
  cash_in: string;
  refunds_out: string;
  cost_incurred: string;
  net_cash: string;
}

export interface CashFlowTotals {
  cash_in: string;
  refunds_out: string;
  cost_incurred: string;
  net_cash: string;
}

export interface CashFlowReport {
  start_date: string;
  end_date: string;
  months: CashFlowMonth[];
  totals: CashFlowTotals;
  cost_incurred_caveat: string;
}

// --- API error shape (see app/api/errors.py) ---
// FastAPI's default HTTPException body: {"detail": "<message>" | [...]}
export interface ApiErrorBody {
  detail?: string | { msg: string; loc?: (string | number)[] }[];
}

// --- Field app: attachments + time clock (Phase 12) ---
// Mirrors app/schemas/field_app.py exactly.

export type JobAttachmentKind = "photo" | "signature" | "other";

export interface JobAttachment {
  id: string;
  job_id: string;
  kind: JobAttachmentKind;
  content_type: string;
  uploaded_by: string;
  created_at: string;
  data: string | null;
}

export interface JobAttachmentInput {
  kind: JobAttachmentKind;
  data: string;
  content_type?: string;
  idempotency_key?: string | null;
  /** Original device enqueue time (ISO). Forwarded for server age audit. */
  client_queued_at?: string | null;
```

```
}

export interface JobTimeEntry {
  id: string;
  job_id: string;
  technician_id: string;
  clocked_in_at: string;
  clocked_out_at: string | null;
  created_at: string;
  updated_at: string;
}

export interface ClockActionInput {
  idempotency_key?: string | null;
  /** Original device enqueue time (ISO). Forwarded for server age audit. */
  client_queued_at?: string | null;
}

// --- inventory, vendors & purchase orders (Phase 13) ---

export interface InventoryItem {
  id: string;
  company_id: string;
  name: string;
  sku: string | null;
  unit_cost: string;
  retail_price: string;
  currency: string;
  quantity_on_hand: number;
  reorder_point: number;
  low_stock_alerted: boolean;
  default_vendor_id: string | null;
  created_at: string;
}

export interface InventoryItemInput {
  name: string;
  sku?: string | null;
  unit_cost?: string;
  retail_price?: string;
  reorder_point?: number;
  default_vendor_id?: string | null;
}

export interface ReorderSuggestion {
  id: string;
  name: string;
  sku: string | null;
  quantity_on_hand: number;
  reorder_point: number;
  default_vendor_id: string | null;
  unit_cost: string;
}

export interface GeneratePOInput {
  vendor_id: string;
  item_ids: string[];
}

export interface Vendor {
  id: string;
  company_id: string;
  name: string;
  contact_email: string | null;
  contact_phone: string | null;
  notes: string | null;
  is_active: boolean;
  created_at: string;
  updated_at: string;
}

export interface VendorInput {
  name: string;
  contact_email?: string | null;
```

```
  contact_phone?: string | null;
  notes?: string | null;
}

export type PurchaseOrderStatus = "draft" | "submitted" | "received" | "cancelled";

export interface PurchaseOrderLineItem {
  id: string;
  inventory_item_id: string;
  quantity_ordered: number;
  quantity_received: number;
  unit_cost: string;
}

export interface PurchaseOrderLineItemInput {
  inventory_item_id: string;
  quantity_ordered: number;
  unit_cost?: string;
}

export interface PurchaseOrder {
  id: string;
  company_id: string;
  vendor_id: string;
  status: PurchaseOrderStatus;
  created_by: string;
  submitted_at: string | null;
  received_at: string | null;
  notes: string | null;
  created_at: string;
  updated_at: string;
  line_items: PurchaseOrderLineItem[];
}

export interface PurchaseOrderInput {
  vendor_id: string;
  notes?: string | null;
  line_items: PurchaseOrderLineItemInput[];
}

export interface ReceiptLineInput {
  line_item_id: string;
  quantity: number;
}

export interface ReceivePurchaseOrderInput {
  receipts: ReceiptLineInput[];
}

// ---------------------------------------------------------------------------
// Marina / slip management (Phase 15)
// ---------------------------------------------------------------------------
export type SlipType = "wet_slip" | "dry_stack" | "mooring";
export type SlipStatus = "available" | "occupied" | "reserved" | "maintenance";

export interface Slip {
  id: string;
  company_id: string;
  identifier: string;
  slip_type: SlipType;
  status: SlipStatus;
  length_ft: string | null;
  width_ft: string | null;
  depth_ft: string | null;
  rack_level: number | null;
  rack_position: string | null;
  latitude: string | null;
  longitude: string | null;
  monthly_rate: string;
  daily_rate: string;
  notes: string | null;
  created_at: string;
  updated_at: string;
}
```

```ts
export interface SlipInput {
  identifier: string;
  slip_type: SlipType;
  status?: SlipStatus;
  length_ft?: string | null;
  width_ft?: string | null;
  depth_ft?: string | null;
  rack_level?: number | null;
  rack_position?: string | null;
  latitude?: string | null;
  longitude?: string | null;
  monthly_rate?: string;
  daily_rate?: string;
  notes?: string | null;
}

export type SlipReservationStatus =
  | "pending"
  | "confirmed"
  | "checked_in"
  | "checked_out"
  | "cancelled";

export interface SlipReservation {
  id: string;
  company_id: string;
  slip_id: string;
  customer_id: string;
  vessel_id: string | null;
  status: SlipReservationStatus;
  start_date: string;
  end_date: string;
  checked_in_at: string | null;
  checked_out_at: string | null;
  cancelled_at: string | null;
  notes: string | null;
  created_at: string;
  updated_at: string;
}

export interface SlipReservationInput {
  slip_id: string;
  customer_id: string;
  vessel_id?: string | null;
  start_date: string;
  end_date: string;
  notes?: string | null;
}

export interface SlipAvailability {
  slip_id: string;
  start_date: string;
  end_date: string;
  available: boolean;
}

export interface GenerateStorageChargeInput {
  rate?: string | null;
  quantity?: string | null;
  description?: string | null;
}

export interface StorageCharge {
  id: string;
  job_id: string | null;
  slip_reservation_id: string | null;
  kind: string;
  description: string;
  quantity: string;
  unit_price: string;
  line_total: string;
  taxable: boolean;
  invoice_id: string | null;
```

```
  invoiced_at: string | null;
  created_at: string;
  updated_at: string;
}

// ===== FILE: frontend/src/vite-env.d.ts =====
/// <reference types="vite/client" />
/// <reference types="vite-plugin-pwa/client" />
```